

The implementation solution for automatic visualization of tabular data in relational databases based on large language models

Hao Yang

*Beijing Advanced Innovation Center for language Resources
Beijing Language and Culture University
Beijing, China
howyoung80@163.com*

Zhaoyong Yang

*Beijing Advanced Innovation Center for language Resources
Beijing Language and Culture University
Beijing, China
yangzhaoyong_blcu@163.com*

Ruyang Zhao

*Beijing Advanced Innovation Center for language Resources
Beijing Language and Culture University
Beijing, China
1956731624@qq.com*

Xiaoran Li

*Beijing Advanced Innovation Center for language Resources
Beijing Language and Culture University
Beijing, China
ella.li68@yahoo.com*

Gaoqi Rao

*Research Institute of International Chinese Language Education
Beijing Language and Culture University
Beijing, China
raogaoqi@blcu.edu.cn*

Abstract—In the data analysis process, visualized data can help users gain better insights. To make it easier and faster for users to obtain visual charts from data, natural language interfaces for data visualization have emerged. Users only need to provide the visualization model with the data to be visualized and a description of their visualization needs, and the model will return a visual chart(NL2VIS). In real-world scenarios, most data is stored in relational databases. To visualize this data, it is first necessary to generate a structured query statements based on the user's visualization requirements(NL2SQL), and then proceed with the subsequent visualization operations. This study breaks down the task of automatic visualization of tabular data in relational databases into three main steps: generating SQL, determining the chart type, and mapping data to visual channels. We utilize the Chain-of-Thought(CoT) technique of generative large language models to address the task of automatic visualization of tabular data. Finally, we evaluated our approach on the nvBench dataset, and the results show that CoT-based automatic visualization of tabular data performs well.

Index Terms—NL2VIS, NL2SQL, Chain-of-Thought, Large Language Model

I. INTRODUCTION

The visualization of tabular data is extremely important in data analysis. A good chart not only clearly presents data characteristics but also helps users gain insights into data patterns. To assist users in automatically visualizing tabular data, a series of commercial software solutions have emerged, such as

Power BI [1], Redash [23], Tableau [24], etc. Users can utilize these software tools to visualize their data through interface selections, drag-and-drop operations, and other interactions. Although these tools can help users visualize tabular data, they require users to have professional data analysis skills and visualization knowledge, which creates a relatively high barrier to entry. Therefore, the academic and industrial communities have begun researching natural language interfaces for tabular data visualization, aiming to achieve automatic visualization of tabular data. The goal is to automatically generate a chart based on the user's natural language description.

Research on automatic visualization of tabular data has generally gone through three main stages: rule-based stage [2], [5], deep neural network-based stage [8], [9], and large language model-based stage [12], [15]. Its generalization ability and robustness have gradually increased with the iterative upgrading of technology. From the perspective of model output, previous research on automatic visualization of tabular data can mainly be divided into two categories. One is to output executable visualization language scripts, and the other is to output abstract expressions. The method of outputting visualization language scripts directly utilizes generative language models to generate target code, such as Vega-Lite [13], Python [11], etc. The method of outputting abstract expressions mainly involves the model generating a predefined visualization query statement, such as Vega-Zero (as is illustrated in the Fig. 1) [9], DVQ (as is illustrated in the Fig. 2) [21], etc., which is then converted into a visualization language program. Unlike

Supported by NSFC(No. 62076038), achievements of the Project of Intelligent International Chinese Education at Beijing Language and Culture University

these two mainstream approaches, in this study, we need the model to output SQL.

```
mark bar
data document_locations
encoding
x location_code
y aggregate count location_code
transform
group x
sort y desc
```

Fig. 1: Vega-Zero abstract statement expression.

```
Visualize BAR
SELECT Date_Claim_Settled ,
COUNT(Date_Claim_Settled)
FROM Settlements
ORDER BY COUNT(Date_Claim_Settled)
DESC
BIN Date_Claim_Settled BY YEAR
```

Fig. 2: Data Visualization Query(DVQ) abstract statement expression.

To visualize tabular data in relational databases, data acquisition is the first step. Retrieving data from relational databases relies on the execution of SQL statements by the database engine. Therefore, to visualize tabular data in relational databases, we must first generate SQL queries based on the user’s natural language description. Subsequently, we proceed with chart selection and mapping data to visual channels.

Large language models like GPT-3.5 have demonstrated powerful text generation and semantic understanding capabilities, achieving state-of-the-art performance in many downstream tasks. Prompt techniques using large language models, such as CoT, imitate human problem-solving approaches by step-by-step reasoning, and have been widely researched by scholars. In this study, we will utilize CoT prompt techniques to address the task of automatic visualization of tabular data in relational databases (CoT-VIS).

II. RELATED WORK

In order to facilitate data analysis and visualization, a series of tools such as Power BI [1], Tableau [24], and Redash

[23] have emerged successively. Users can utilize these tools for data visualization, but they often require a significant amount of manual operation. Subsequently, both academic and industry began researching natural language interfaces for data visualization, making user operations more convenient. These natural language interfaces enable users, even those lacking data analysis and visualization experience, to generate charts through language descriptions. The research on natural language interfaces for visualization has gone through three stages: rule-based methods, such as NL4DV [2], Orko [3], Eviza [4], FlowSense [5], DataTone [6]; deep neural network-based methods, such as ADVisor [7], Seq2vis [8], ncNet [9], RGVisNet [10]; and generative large language model based methods, such as Chat2Vis [11], Prompt4Vis [12], Mirror [13], ChartGPT [14], and LIDA [15]. At present, the development of automated data visualization has progressed to a stage where large-scale models are used to solve problems. Based on large language models, two mainstream solutions have emerged: directly generating target code and visualization abstract expressions. For example, Chat2Vis [11] directly generates Python programs, while Prompt4Vis [12] generates visualization abstract expressions. To further improve the accuracy of data visualization, researchers tend to decompose visualization tasks into several sub-tasks, with each subtask responsible for solving a specific problem. The results of each subtask are then inputted into subsequent sub-tasks. For example, both ChartGPT [14] and Prompt4Vis [12] decompose the visualization task into sub-tasks and then solve each sub-task one by one, ultimately achieving good results in completing the visualization task.

Large language models possess powerful In-Contextual Learning abilities [16]. In order to further enhance their reasoning capabilities, researchers have investigated prompt techniques such as zero-shot [17] and few-shot [16]. Subsequently, researchers discovered a Chain-of-Thought [18] prompt approach, which mimics the human thinking process by solving problems step by step. In academia, based on the chain-of-thought, other variants have been studied, such as Contrastive Chain-of-Thought [19], and Least-to-Most [20], etc.

III. DEFINITION OF PROBLEM

When solving problems using prompt-based techniques with generative large language models, users need to provide the model with a prompt for problem solving. The model then performs inference based on the prompt provided by the user and generates a response to the problem. We define the large language model as LLM , the prompt provided by the user as P , and the model’s response as A . This process can be formalized as follows:

$$LLM(P) \rightarrow A$$

The automatic visualization of tabular data is the process by which a model provides users with a chart based on natural language descriptions and database schema information. Generating a chart typically requires three types of information:

the chart type, the data, and the mapping between the data and visual channels. Therefore, it is necessary to assemble the user's natural language descriptions and the database schema information into a prompt as input to the LLM. The data objects in this study are tabular data in relational databases. The model needs to provide structured query statements, chart types, and mappings between data and visual channels based on the user's natural language descriptions and the database schema information. The process of automatic visualization of tabular data in relational databases is illustrated in the Fig. 3. We define the user's natural language descriptions as NL , the database schema information as D , structured query statements as SQL , chart types as $CHART$, and the mapping between data and visual channels as MAP . This process can be formalized as follows:

$$LLM(P(NL, D)) \rightarrow \{SQL, CHART, MAP\}$$

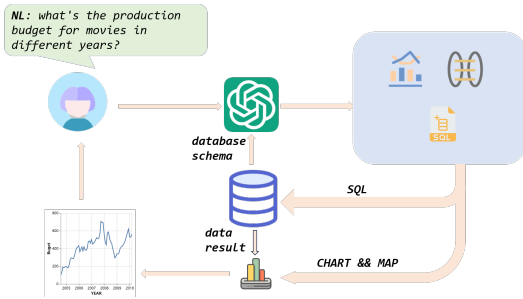


Fig. 3: The workflow of automatic visualization of tabular data in relational databases .

IV. SOLUTION OF CoT-VIS

To enhance the performance of large language models on downstream tasks, there are typically two mainstream approaches: first, fine-tuning the model on a dataset specific to the downstream task; second, constructing prompts to leverage the model's In-Context Learning (ICL) ability to solve problems. Fine-tuning a model demands significant computational power and high-quality downstream task datasets, making it highly resource-intensive. In contrast, prompt engineering is simple and easy to implement, as it does not require updating the model's parameters. A well-crafted prompt can guide the model to generate highly accurate answers. Prompt engineering is currently a subject of extensive research in the academic community and has already achieved remarkable success in tasks such as commonsense reasoning, mathematical problem-solving, and symbolic reasoning [18].

Next, we will elaborate on how to use prompt techniques in large language models to address the task of automatic visualization of tabular data in relational databases. First, we will summarize the current prompt strategies employed to enhance the reasoning capabilities of large language models. Then, we will select an appropriate solution specifically for the task of automatic visualization of tabular data.

A. Prompt Strategies for Large Language Models

Large language models can leverage their ICL abilities to respond to user queries based on the provided prompt text. Depending on the number of examples given by the user, prompt strategies can be categorized into two types: zero-shot and few-shot styles. Zero-shot prompts do not provide the model with question-answer examples, requiring the model to respond directly to the query. Few-shot prompts, on the other hand, provide the model with several question-answer examples, which can further enhance the accuracy of the model's responses.

The user provides the model with question-answer pairs in the form $\langle q, a \rangle$. Here, q represents the user's question along with any additional information needed to solve the problem, and a represents the model's response. Typically, prompts in the $\langle q, a \rangle$ format are referred to as standard-prompt. Researchers have discovered that by adding reasoning steps r , prompting the model to derive the answer step by step, the accuracy of the model's responses can be significantly improved. Consequently, researchers proposed the $\langle q, r, a \rangle$ format prompt and named it Chain-of-Thought (CoT) prompt. Usually, users need to manually create several CoT examples to enable the model to mimic this format to generate answers. This type of prompt is known as few-shot CoT prompting. Additionally, there is a method that does not require manually crafted prompts: by providing the model with "Let's think step by step.", the model is induced to produce CoT responses. This is referred to as zero-shot CoT prompting. Empirical evidence shows that manually few-shot CoT prompts outperform zero-shot CoT prompts in terms of performance. CoT prompt breaks down problems step by step, solving them gradually, which enhances problem-solving capabilities compared to standard prompts.

B. CoT-VIS

As previously mentioned, visualizing tabular data in a relational database involves three major steps: generating SQL, determining the chart type, and mapping data to visual channels. We will now detail these three steps, followed by developing a CoT prompt based on this process.

1) *Generating SQL*: The model needs to generate the corresponding SQL based on the user's natural language description to query data. Unlike traditional NL2SQL tasks, in a visualization task, it is not only necessary to identify the correct data columns and tables but also to perform appropriate data transformations on the data columns, such as data binning.

a) *Step 1: Determining Data Columns*: After semantically understanding the user's natural language description and the database schema, the language model determines the data columns that need to be queried. Assuming there are n data columns $\{column_1, column_2, \dots, column_n\}$ in the database, this process will ultimately yield an intermediate SQL expression in the following form:

```
SELECT columni | columnj | ...
```

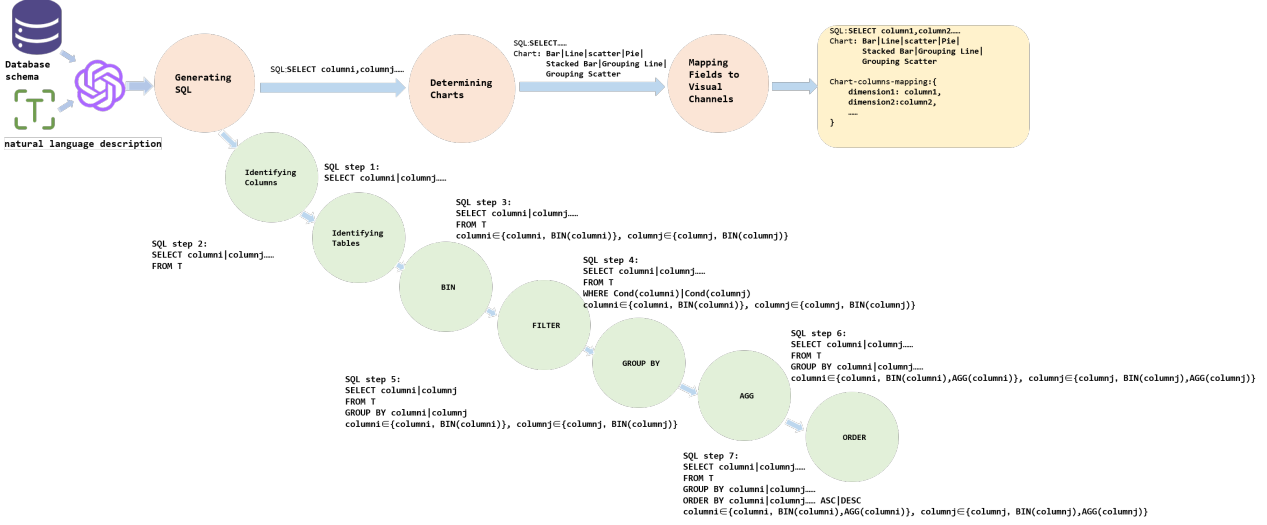


Fig. 4: Chain-of-thought deduction process for automatic visualization of tabular data in relational databases.

b) *Step 2: Determining Data Tables:* Based on step 1, determine the data tables according to the selected data columns. If the data columns come from a single table T_i , then the target data $T = T_i$. If the selected columns come from multiple tables $\{T_1, T_2, \dots, T_n\}$, it is necessary to perform multi-table joins based on foreign key information. This process will generate an intermediate SQL expression in the following form:

```
SELECT columni | columnj | ...
FROM T
```

where $T = T_i \mid JOIN(T_i, T_j, \dots \mid \text{Foreign Keys})$.

c) *Step 3: Data Transformation:* To determine whether transformation operations are needed for the selected columns, primarily data binning operations. Many previous works relied on the data binning functionality inherent in frontend visualization frameworks (such as vega-lite). In this study, data is obtained through generating SQL queries, and SQL itself supports data transformation operations. Specifically, data binning operations are typically implemented using the `CASE` expression in SQL. If there is a field named *timestamp*, representing timestamps, and the user requires data to be binned based on early morning, morning, afternoon, and evening time periods, the `CASE` expression can be used to achieve this:

```
SELECT
CASE
WHEN strftime('%H', timestamp)
BETWEEN '00' AND '05' THEN '
early morning'
WHEN strftime('%H', timestamp)
BETWEEN '06' AND '11' THEN '
morning'
WHEN strftime('%H', timestamp)
BETWEEN '12' AND '17' THEN '
afternoon'
WHEN strftime('%H', timestamp)
BETWEEN '18' AND '23' THEN '
evening'
ELSE 'unknown'
END AS bucket
FROM
T;
```

Almost all database engines support basic binning operations for time or numeric data. After the data has been binned, we will obtain the following intermediate SQL:

```
SELECT columni | columnj | ...
FROM T
```

where, $column_i \in \{column_i, BIN(column_i)\}, column_j \in \{column_j, BIN(column_j)\} \dots$

d) *Step 4: Data Filtering:* Based on the user's natural language description, determine the data filtering conditions to obtain the data that meets the user's requirements. In SQL, data filtering is done through the `WHERE` clause. After this step, we will obtain an intermediate SQL in the following form (Cond represents the condition filtering operation):

```
SELECT columni | columnj | ...
FROM T
WHERE Cond(columni) | Cond(columnj) | ...
```

where, $column_i \in \{column_i, BIN(column_i)\}, column_j \in \{column_j, BIN(column_j)\} \dots$

e) *Step 5: Data Group By:* Determine whether to perform a grouping operation (GROUP BY) based on certain data columns. After binning and grouping operations, the data will be divided into different groups based on values. After this step, we will obtain an intermediate SQL in the following form:

```
SELECT columni | columnj | ...
```

FROM T
GROUP BY $column_i \mid column_j \mid \dots$

where, $column_i \in \{column_i, BIN(column_i)\}$, $column_j \in \{column_j, BIN(column_j)\} \dots$

f) *Step 6: Data Aggregation:* After dividing the data into different groups, it is common to perform aggregation (AGG) operations on the data within the same group. Basic column aggregation operations include SUM(), COUNT(), AVG(), etc. After the data aggregation operation, we will obtain the following intermediate SQL:

SELECT $column_i \mid column_j \mid \dots$
FROM T
GROUP BY $column_i \mid column_j \mid \dots$

where,

$column_i \in \{column_i, BIN(column_i), AGG(column_i)\}$,
 $column_j \in \{column_j, BIN(column_j), AGG(column_j)\}$

g) *Step 7: Data Order By:* The final step in generating the target SQL is to determine whether certain columns need to be sorted in ascending or descending order. In visualization charts, it is very common to sort by the names on the x-axis, for example. After this step, we will obtain the final SQL expression:

SELECT $column_i \mid column_j \mid \dots$
FROM T
GROUP BY $column_i \mid column_j \mid \dots$
ORDER BY $column_i \mid column_j \mid \dots$ ASC|DESC
where,

$column_i \in \{column_i, BIN(column_i), AGG(column_i)\}$,
 $column_j \in \{column_j, BIN(column_j), AGG(column_j)\}$

2) *Determining the Type of Chart:* The language model will determine the type of chart based on the user's natural language description and the generated SQL. Our data visualization system supports seven types of charts: Scatter, Pie, Bar, Stacked Bar, Line, Grouping Scatter, and Grouping Line.

3) *Mapping:* Mapping the target fields of the SQL query to the visual channels of the data chart. For example, a bar chart has two dimensions: the x-axis and the y-axis. If the SQL query fields are $column_1$ and $column_2$, where $column_1$ corresponds to the x-axis and $column_2$ corresponds to the y-axis of the bar chart. After reasoning based on contextual information, the language model will obtain: $\{\text{'x-axis': 'column_1'}, \text{'y-axis': 'column_2'}\}$.

After generating SQL, determining the chart type, and mapping the visual channels, the model will output all the information needed for visualization through chain-of-thought reasoning. The complete deductive process is illustrated in the Figure 4.

V. EXPERIMENT

a) *Dataset:* We chose nvBench [8] as the evaluation dataset, widely used in the field of data visualization. nvBench is a large dataset designed for complex and cross-domain NL2VIS tasks, covering 105 domains, supporting seven common types of visualizations (Bar, Line, Scatter, Pie, Stacked Bar, Grouping Line, Grouping Scatter) and containing 25,750 (NL, VIS) pairs. Following the experimental validation method outlined in the Prompt4Vis paper [12], we randomly selected 141 databases from nvBench and divided them into training, validation, and test sets in a ratio of 7:2:1. Specifically, the training set contains 98 databases, the validation set contains 14 databases, and the test set contains 29 databases.

b) *Model and Methods:* In this experiment, we selected the GPT-3.5-turbo model interface provided by OpenAI and set temperature=0. The comparative methods include: zero-shot prompt, few-shot prompt, zero-shot-CoT prompt, and few-shot-CoT prompt. In the zero-shot prompt method, the model directly responds based on the provided database schema information and user natural language description. The few-shot prompt method requires manually writing several examples to provide to the model before letting it respond. The Zero-shot-CoT prompt method adds "Let's think step by step." to induce the model to generate chain-of-thought responses based on the zero-shot prompt. The few-shot-CoT prompt method involves manually writing seven chain-of-thought examples (one for each type of chart) to provide to the model for its response.

c) *Metrics:* This study comprehensively evaluates the data accuracy (data acc), axis accuracy(axis acc), chart accuracy(chart acc), and overall accuracy(overall acc) of the data visualization system. Data accuracy refers to the execution accuracy of the SQL predicted by the model, calculated as the proportion of correctly executed results to the total number of results. Axis accuracy is the accuracy of mapping data to visual channels, calculated similarly as a proportion. Chart accuracy compares the predicted chart type by the model with the gold chart, also calculated as a proportion.

Method	Data acc	Axis acc	Chart acc	Overall acc
zero-shot	0.304	0.371	0.759	0.217
few-shot	0.501	0.497	0.928	0.326
zero-shot-CoT	0.490	0.494	0.939	0.274
few-shot-CoT	0.559	0.810	0.975	0.490

TABLE I: Experiment result

d) *Experiment Result:* Experimental results show that with zero-shot and zero-shot-CoT prompt methods, large language models struggle with the automatic visualization of tabular data, exhibiting very low accuracy. Providing the model with a few examples using few-shot prompts can improve the model's accuracy to some extent. Our designed few-shot-CoT prompts significantly enhance the model's response accuracy. This proves that the chain-of-thought prompts designed for the automatic visualization of tabular data in relational databases are highly effective. However, we must also note that there is still considerable room for improvement in accuracy, which

inspires us to design more sophisticated deduction algorithms to further enhance accuracy.

VI. CONCLUSION

This study divides the task of automatic visualization of tabular data in relational databases into three main steps: generating SQL, determining the chart type, and mapping data to visual channels. Using the Chain-of-Thought technique of large language models, we perform step-by-step reasoning for these three steps. Experimental validation demonstrates that the Chain-of-Thought technique can be effectively applied to the task of automatic data visualization, significantly improving its accuracy. However, we should also note that there is still considerable room for improvement in the accuracy of automatic data visualization tasks, and we will continue to explore ways to enhance this accuracy. Additionally, for the task of automatic visualization of tabular data, the only evaluation dataset available is nvBench. We look forward to the academic community producing more high-quality evaluation datasets.

REFERENCES

- [1] Louis T Becker and Elyssa M Gould. 2019. Microsoft power BI: extending excel to manipulate, analyze, and visualize diverse data. *Serials Review* 45, 3 (2019), 184–188.
- [2] Arpit Narechania, Arjun Srinivasan, and John Stasko. 2020. NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries. *IEEE Transactions on Visualization and Computer Graphics* 27, 2 (2020), 369–379.
- [3] Arjun Srinivasan and John Stasko. 2017. Orko: Facilitating multimodal interaction for visual exploration and analysis of networks. *IEEE transactions on visualization and computer graphics* 24, 1 (2017), 511–521.
- [4] Vidya Setlur, Sarah E Battersby, Melanie Tory, Rich Gossweiler, and Angel X Chang. 2016. Eviza: A natural language interface for visual analysis. In *Proceedings of the 29th annual symposium on user interface software and technology*. 365–377.
- [5] Bowen Yu and Cláudio T Silva. 2019. FlowSense: A natural language interface for visual data exploration within a dataflow system. *IEEE transactions on visualization and computer graphics* 26, 1 (2019), 1–11.
- [6] Tong Gao, Mira Dontcheva, Eytan Adar, Zhicheng Liu, and Karrie G Karahalios. 2015. Datatone: Managing ambiguity in natural language interfaces for data visualization. In *Proceedings of the 28th annual acm symposium on user interface software & technology*. 489–500.
- [7] Can Liu, Yun Han, Ruike Jiang, and Xiaoru Yuan. 2021. Advisor: Automatic visualization answer for natural-language question on tabular data. In *2021 IEEE 14th Pacific Visualization Symposium (PacificVis)*. IEEE, 11–20.
- [8] Yuyu Luo, Nan Tang, Guoliang Li, Chengliang Chai, Wenbo Li, and Xuedi Qin. 2021. Synthesizing natural language to visualization (NL2VIS) benchmarks from NL2SQL benchmarks. In *Proceedings of the 2021 International Conference on Management of Data*. 1235–1247.
- [9] Yuyu Luo, Nan Tang, Guoliang Li, Jiawei Tang, Chengliang Chai, and Xuedi Qin. 2021. Natural language to visualization by neural machine translation. *IEEE Transactions on Visualization and Computer Graphics* 28, 1 (2021), 217–226.
- [10] Yuanfeng Song, Xuefang Zhao, Raymond Chi-Wing Wong, and Di Jiang. 2022. Rgvisnet: A hybrid retrieval-generation neural framework towards automatic data visualization generation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1646–1655.
- [11] Paula Maddigan and Teo Susnjak. 2023. Chat2vis: Generating data visualisations via natural language using chatgpt, codex and gpt-3 large language models. *Ieee Access* (2023).
- [12] Shuaimin Li, Xuanang Chen, Yuanfeng Song, Yunze Song, and Chen Zhang. 2024. Prompt4Vis: Prompting Large Language Models with Example Mining and Schema Filtering for Tabular Data Visualization. *arXiv preprint arXiv:2402.07909* (2024).
- [13] Canwen Xu, Julian McAuley, and Penghan Wang. 2023. Mirror: A Natural Language Interface for Data Querying, Summarization, and Visualization. In *Companion Proceedings of the ACM Web Conference 2023*. 49–52.
- [14] Yuan Tian, Weiwei Cui, Dazhen Deng, Xinjing Yi, Yurun Yang, Haidong Zhang, and Yingcai Wu. 2024. Chartgpt: Leveraging llms to generate charts from abstract natural language. *IEEE Transactions on Visualization and Computer Graphics* (2024).
- [15] Victor Dibia. 2023. LIDA: A Tool for Automatic Generation of Grammar-Agnostic Visualizations and Infographics using Large Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Danushka Bollegala, Ruihong Huang, and Alan Ritter (Eds.). Association for Computational Linguistics, Toronto, Canada, 113–126. <https://doi.org/10.18653/v1/2023.acl-demo.11>
- [16] Brown T, Mann B, Ryder N, et al. Language models are few-shot learners[J]. *Advances in neural information processing systems*, 2020, 33: 1877-1901.
- [17] Wei J, Bosma M, Zhao V Y, et al. Finetuned language models are zero-shot learners[J]. *arXiv preprint arXiv:2109.01652*, 2021.
- [18] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models[J]. *Advances in neural information processing systems*, 2022, 35: 24824-24837.
- [19] Chia Y K, Chen G, Tuan L A, et al. Contrastive chain-of-thought prompting[J]. *arXiv preprint arXiv:2311.09277*, 2023.
- [20] Zhou D, Schärli N, Hou L, et al. Least-to-most prompting enables complex reasoning in large language models[J]. *arXiv preprint arXiv:2205.10625*, 2022.
- [21] Luo Y, Qin X, Tang N, et al. Deepeye: Towards automatic data visualization[C]//2018 IEEE 34th international conference on data engineering (ICDE). IEEE, 2018: 101-112.
- [22] Satyanarayan A, Moritz D, Wongsuphasawat K, et al. Vega-lite: A grammar of interactive graphics[J]. *IEEE transactions on visualization and computer graphics*, 2016, 23(1): 341-350.
- [23] Leibzon A, Leibzon Y. Redash V5 Quick Start Guide: Create and Share Interactive Dashboards Using Redash[M]. Packt Publishing Ltd, 2018.
- [24] Batt S, Grealis T, Harmon O, et al. Learning Tableau: A data visualization tool[J]. *The Journal of Economic Education*, 2020, 51(3-4): 317-328.